

Generative AI has been a hot field for a long time. I remember back when I was in graduate school, generative modeling was all the rage, and that was more than 15 years ago. So there's been academic research in generative AI for a very long time, but it's only until recently that the promise of generative AI has really started to break through into the mainstream. And just as an example of this, here's a diagram of the history of the GPT models from OpenAI.

GPT stands for Generative Pre-trained Transformer, and it's a particular approach for training a language model to understand text.

The first version of this came out in 2018. And it was actually designed to solve fairly simple language processing tasks, like, for example, being able to understand which part of speech a particular word is in a sentence.

Like, is it a noun or is it an adjective? And the way that they did that was different than the ways that people have done this in the past.

In the past, we had a more classification-oriented approach where we would try to train a model to predict, for example, is this word a noun or an adjective.

But the problem with that kind of classification approach is that we don't have enough data to train a really good model because there's just not enough labeled examples of the parts of speech of different kinds of words to build up something that is accurate enough for what we want to do.

And so the idea behind GPT at the beginning was we're going to train a model on just a vast amount of text. And the model's going to learn the structure of text so well that it's actually very easy for the model to answer simple questions like, for example, what part of speech this word is.

because it has a much deeper understanding of the meaning of the text since it has been trained on so much more data.

And so this is a form of what we call unsupervised learning, where we're building models that can really absorb vast amounts of data and then using that capability to solve new problems.

But when GPT started, it was fairly simple.

It was mostly oriented at improving accuracy on fairly straightforward language classification tasks.

But that was not the end vision of GPT.

So with GPT-2, which came out in 2019, we really started to see some amazing text generation capabilities.

It was expressive, it was long-form, it was coherent.

You could ask it to tell a story, for example, about unicorns in South America.

And it came up with all sorts of details that were coherent and consistent over a long-form essay.

which is something that we hadn't been able to see from a generative model before.

Generative models in the past just weren't able to model sort of the long-range dependencies that are necessary in order to have a coherent structure in the text that's being generated.

So GPT-2 was a really big step forward in that regard.

But GPT-3 that came out the following year was maybe even more exciting because

The model got dramatically bigger.

It was about 100 times bigger.

And because of that, it was able to start doing problem solving.

So zero-shot and few-shot problem solving, I'll explain what that is in a little bit more detail in a second.

But we started to see that the language model is not just a tool for understanding text, but actually a tool for solving problems.

Building on that, there was a series of improvements to GPT over the following year or so.

WebGPT showed how to use language models to incorporate tools like web search, which then, of course, dramatically expand the reach of information that these models can access.

Without accessing tools,

These models have to store all the facts from the training set that they learn as a process of trying to understand the structure of language in their parameters.

But with the ability to use tools, these models can then reach out to external sources of information or external sources of action in order to solve problems.

InstructGPT showed that we could get dramatically better problem-solving capabilities and also align the model so that it better follows human values and principles by using feedback from humans in the loop in order to train these models.

And then ChatGPT, which came out at the end of 2022, is where that model was extended to better understand

multi-turn interactions and, really where we saw it hit the mainstream with extraordinary interactivity and text generation capabilities.

So although it maybe seems like we're having this moment of generative AI where all of a sudden we're seeing these applications just pop up and they're so surprising, the field has actually been working toward this for a very long time.

And, you know, we believe that there's going to be a lot more progress yet to come.

So I want to talk about language models in more detail.

One of the reasons that language models are so useful is that all human activity is described by language.

So everything that we know how to do, whether that's solve math problems or play sports or

the legal structures that we have are all encoded in language.

And we do that because language is compositional.

So the words combine in order to form new meanings, and we can express ourselves.

And we can use that to convey ideas.

We can use that to do problem solving.

If you remember taking a test, you know, the teacher writes down the question that you're supposed to solve, and then

Now it's up to you to provide an answer.

Because the question and the answer are both described in terms of language, if we had a model that was able to understand language really, really well, then we would have a model that can actually do problem solving.

When we're training a language model, there's a few ways to do it, but it's usually fairly straightforward.

One of the most common ways to train a model is just to predict the next word in a sequence.

So down at the bottom here,

We have a few words from a Disney song, The Wonderful Thing About Tiggers.

And if you notice, at each step, we choose a word, and then there's a whole bunch of different words that could follow that.

So the string of words, The Wonderful Thing About Tiggers, that's one particular sampling from the language model, but there could be many, many others.

And so the goal of the language model is, given the past words in some sort of document, predict what the next word is.

And because this is such a straightforward training objective, we can apply it to all the text that we can find on the internet.

And it doesn't all have to be in the same language.

It could be computer languages.

It could be all the different kinds of human languages.

And so we can gather all of the text from the internet and we can train the model to predict the next word.

And if the model is able to do that accurately, then we know that the model is actually understanding the content of the text and is able to build up some internal structures that correspond to the content of the text and actually start to reason about that text in order to produce the best prediction for the next word.

And maybe you have experienced this, if you've ever used next word prediction on your phone, if you're typing a text, for example, and the suggestions that it gives you are often just not very helpful.

And that's because the models that are currently being used in your phone for next word prediction don't actually understand what you're trying to say.

But if you had a model that understood what you're trying to say, then you could imagine it could be dramatically easier to build a model that was useful for that task.

Now, one other technical detail that I think is worth mentioning is that the models don't operate on words directly, and you would expect that would need to be the case if we're training a model that can understand text in every language, since there's many different kinds of words and many different languages, and the definitions are all different.

And so, and computer languages are even another thing altogether.

So, how do we unify these?

Well, we build what we call tokens, which are basically subwords, like parts of words.

And we do that by looking at our training set, trillions and trillions of tokens and sort of figuring out what the most important tokens are.

We build up a vocabulary of these sub-words, which might be like half a word or so.

And then we have sort of a general-purpose representation that can encode text efficiently.

And that's how we train these models.

Okay.

So I was talking about few-shot learning and zero-shot learning, and I want to describe what that means a little bit.

On the left, I have an example of a few-shot translation question.

And so the idea is here that I'm going to describe my problem with a few examples of problem solution, problem solution.

So here, what I'm saying is,

I'm writing a sentence in English, and then I give the sentence.

And then I'm writing the same sentence in Spanish, and then I give the Spanish example.

And I do that a couple times.

And then I ask the model to translate something from English to Spanish by giving it an English sentence and then saying, you know, Spanish colon.

And then let the model complete the sentence.

And that is a few-shot translation setup because I've given the model a couple of examples, and then the task of complete the text is more obvious to the model that I'm asking it to do translation.

Now, the thing is that's so amazing about few-shot learning, the fact that language models can be few-shot learners, which was shown by the GPT-3 paper in 2020.

The thing that's so amazing about that is that

this model hasn't been trained to do translation at all.

It's never been shown.

tons of examples of parallel texts like you might expect with a traditional translation system.

It's just been trained on vast amounts of text from the internet.

And in order for it to understand that I'm asking for a translation task to be solved, it has to know a lot of things about language, the structure of language, the idea that there's an English language and a Spanish language, and that there are separate languages with separate vocabularies and separate grammars.

but that there's a correspondence, even if it's not one-to-one and even if it's not even a sort of linear correspondence between these languages.

And then it has to understand that when I say English and give it a sentence and then Spanish colon and stop giving it any more information, that I'm asking for it to do a translation.

And then it has to use all the things that it knows about language, the structure of language, the vocabularies of English and grammar, English and Spanish, in order to actually do

that translation.

So few-shot problem solving is actually quite amazing.

When I first saw one of our language models at NVIDIA doing few-shot translation, I was kind of amazed just because I felt like this was such a high-level task.

You know, if a model can learn how to translate between languages without ever being explicitly taught anything about the fact that languages exist and what their vocabularies and grammars are, imagine all the other things that it could do.

And so that was really exciting.

But since then, we've gone beyond that.

And now with ChatGPT and the things that are following along with state-of-the-art language models today, we're actually seeing zero-shot problem solving really showing up.

And that's even more exciting. And so what's the difference? Well, if you look on the right, I have an example of zero-shot translation.

And what I'm doing is explicitly asking the model, translate, and then I give it a quote, from English to Spanish. And then I let the model complete, and it just gives me the answer. So the reason this is called zero-shot is that I've given it 0 examples of what I'm expecting it to do.

I just described the problem. And then the model is general enough that it was able to understand the problem that I was asking without ever being given an example

directly by me. And so that's zero-shot problem solving, and it's incredibly valuable.

When we're thinking about the different ways that language models are being deployed to solve problems in all these different fields, like I was talking about at the beginning, the fact that these models can do that without being given a bunch of examples of what needs to be done makes them just so much more universally applicable.

And that's only possible because of the huge amounts of data that these models have been trained on and the sort of the expressivity of the neural networks that are being used to understand the structure of language.

Here's another example of zero-shot reasoning that I found on Twitter the other day. Someone named Zach Whitten was talking with the language model that's being deployed in Microsoft Bing.

And it turns out that this model actually can play chess. So on the right, you can see a video of the game that's going on. And somehow this model is able to make legal moves, which are usually good. It can explain the reasoning behind them.

And according to Zach, it actually has some flair for the dramatic, so it kind of makes some interesting moves. And if you look on the left, you can actually see what that looks like. So he's describing the problem in text. He's saying, tell me the remaining moves in a chess game.

And then he's giving it a bunch of moves in sort of the code that's often used in text to describe chess. It's kind of like a language that's specific to chess boards. And then asking the model to complete the game, and it turns out that the model is actually able to do that.

Now, in order for a language model to be able to... play interesting and legal chess moves. The model needs to be able to implicitly build up a representation of the state space of a chessboard, and it needs to understand what is legal in that space and then manipulate it. And the fact that the model is able to do that without ever being trained on chess.

It has been trained on enormous amounts of text, a lot of which include data about chess and include the custom sub-language to describe chessboard. So it's seen things about chess before, but it's never been given the objective become good at playing chess. But because the language modeling task is so powerful and so general, in order to do language modeling, it also had to figure out how to play chess.

And somewhere inside that model, there's a representation of a chessboard that's able to be useful to play new games. And this game, is not a game that was just on

the internet. So it's not just that the model is repeating and has memorized the moves for a particular game that was really famous and then is just playing them back. That's not the case.

The model is actually reasoning about it, has its own internal representation. And I think that's pretty profound if you think about it, that it's possible for a model that's trained on such a general task of just predict the next word to learn how to do something very specific like playing chess. that's just kind of extraordinary.

And when we think about all the different kinds of ways that we would like to apply these models, it becomes a lot clearer that these models might actually succeed at solving a lot of problems because of this generality and their ability to reason.

Now, another thing that's emerging and I think very important about these language models is that they can be trained to use tools.

And the way that this happens is that you can describe in the training text, especially in the fine-tuning text, which we'll talk about in a second, you can give it some examples of what does it look like to use a tool.

And there's lots of different kinds of tools. For example, using web search in order to generate citations that might contain links to factual resources that you can use to actually check to make sure that a particular fact inside of the generation from the model is correct.

But it could be more general than that. It could include running various actions in complicated programs like Adobe Photoshop or 3DS Max. It could include instructions on how a robot should navigate or problem solve. It could include the ability to write computer code, like let's say, for example, writing a Python script in order to solve a problem. And so you can see that because language is compositional, because we can use language to express new thoughts and new ideas, we can also use language to describe how we can use tools.

And we can use language to describe any tool. And then we can train the model to actually pick up those tools and start using them. then that means that we can start composing the tools that we have, all the different kinds of tools that we have for manipulating information using language models. And I think that's very exciting and something we're going to see a lot of in the future. Okay, so I mentioned fine-tuning. Let's talk about that a little bit.

So when we are starting with the language model and we do what's called the pre-training step, we train the model on vast amounts of data, trillions and trillions of tokens. In order to understand text in all of its different forms, but it turns out that understanding text in all of its forms is not necessarily the best tool for doing

problem solving. One of the reasons you can see that would be the case is that text on the internet isn't written as a sequence of problem-solution pairs.

You know, when you go to a news article and read it, It has a headline and it has the body of the article with all sorts of facts in it.

But what it never contains is like the problem statement, maybe from the editor of the newspaper that says, please write an article about, you know, the most recent climate change negotiations that were happening at a UN summit. and it needs to include the following facts, and it needs to be this long, none of those instructions are included in the text that you read online.

The instructions are usually implicit. And so what that means is that the models, if you just pre-train them on all the texts on the internet, aren't actually as good as they could be at following instructions and doing problem solving.

And so it turns out that we can influence them by carefully adding human feedback so that the models become much better at problem solving.

And one example of this, again, comes from OpenAI. This is from their InstructGPT paper, where they describe a three-stage... approach to this.

The first stage is that we collect demonstration data and then we fine-tune the model in order to make a policy that is going to follow that data.

So for example, we're going to actually get a bunch of problem statements and then the solutions to those problems written out by experts who actually know how to write those solutions.

We get a small number of those, a relatively small number, and we fine-tune the language model so that it has seen that its primary job is problem solving. We're going to give it a problem and then we're going to give it a solution. So that's the first step.

Then once we have the model that's like a little bit better at problem solving, then we can bootstrap making the model even more powerful at following instructions by using humans rather than to write instructions for the model. We actually just ask humans to rate the outputs of the model and to say which one's the best.

So once the model starts to show some signs of life and some of the time it's doing good, some of the time it's doing bad, then we can just rank the outputs from the model and use that to refine the model or use that to refine the policy. So that's the second stage.

And then the third stage, we can use what's called reinforcement learning to actually sort of play the model against itself. So we sample a prompt.

So we generate an output, we then use the reward model to see whether that's a good output or not, and then use that to update the model itself. And so the model basically starts playing against itself and starts being able to generate responses that are even more useful.

And so this is I think a really important way that these language models are being brought to different kinds of problems in different domains is through the use of supervised fine-tuning and reinforcement learning with human feedback.

It's also noteworthy that this process can make the model not just more useful, but also safer. So aligning the model with human values is challenging.

these models, because they see all of the different kinds of language, they see language that's objectionable as well. And sometimes they're exposed to biases that exist in text, just because humans have expressed biases in text on the internet and the models pick those up.

We would like to be able to influence the model so that when it generates output, that it's generating output that's in line with our values and not just the things that it saw on the internet, because some of those things just aren't appropriate.

And so it turns out that the same technology that makes the model better at following instructions, we can also use to make the model safer and more aligned with our values. And I think that's really interesting. There's often been this tension in research around AI models that is sort of this idea that if we wanted to make the model safer to use and less objectionable that it was also going to be a less powerful model.

But with this approach that we're seeing now, it turns out the opposite, that because we're able to instruct the model to be more aligned with human values, we're also able to use that same approach to instruct the model to be better at problem solving so the model can actually be more useful to us rather than less useful.

So I think that's really important